

Синхронные и асинхронные модели взаимодействия сервисов

Table of contents

Синхронные и асинхронные модели взаимодействия сервисов	1
Цена синхронного взаимодействия и альтернатива	1
Повторные попытки (Retries)	2
Идемпотентность (Idempotency)	2
Автоматическое размыкание цепи (Circuit Breaker)	2
Передача дедлайна (Deadline Propagation)	3
Зачем асинхронность	3
Брокеры сообщений и очереди	3
Обмен файлами как асинхронный контракт	3
Надёжность и согласованность	4
Постскриптум: критерий архитектурного выбора	4

Синхронные и асинхронные модели взаимодействия сервисов

В синхронной модели взаимодействия клиент отправляет запрос и ожидает ответ в рамках того же пользовательского действия. Для REST, gRPC и GraphQL это базовый режим работы; WebSocket обычно используется для постоянного двустороннего обмена, но также требует управления таймингами и отказами. Синхронная модель удобна, когда нужен мгновенный результат, но именно здесь появляются основные эксплуатационные риски распределённых систем.

Цена синхронного взаимодействия и альтернатива

Когда сервис делает **синхронный** вызов по схеме «запрос-ответ» и ждёт результат, он напрямую зависит от доступности и скорости downstream-сервиса. Практически это означает три базовых класса отказов:

- **Тайм-аут:** зависимость отвечает слишком долго, и вызывающая сторона прерывает ожидание.
- **Неопределённый результат операции:** вызывающая сторона не получила ответ и не знает, была ли операция выполнена на стороне зависимости.
- **Каскадная деградация:** медленный или недоступный downstream удерживает ресурсы upstream-сервисов (пулы соединений, воркеры, потоки), снижая общую пропускную способность системы.

В синхронной модели именно вызывающая сторона (клиент запроса) обычно задаёт `timeout` и стратегию повторов (`retry`), потому что только она знает допустимое время

ожидания в рамках пользовательского сценария и может принять решение: ждать дальше, повторить вызов или вернуть ошибку наверх. Если эти параметры не заданы, зависший downstream начинает удерживать поток/воркер, и при росте нагрузки это быстро приводит к исчерпанию ресурсов.

Упрощённо это выглядит так:

```
// Клиент управляет временем и повторами вызова.
ctx, cancel := context.WithTimeout(parentCtx, 300*time.Millisecond)
defer cancel()

for attempt := 1; attempt <= 3; attempt++ {
    resp, err := paymentsClient.Charge(ctx, req) // синхронный RPC/HTTP-вызов
    if err == nil {
        return resp, nil
    }
    if !isTemporary(err) {
        return nil, err // логическую ошибку не ретраим
    }
    sleep(backoffWithJitter(attempt))
}
return nil, ErrUpstreamUnavailable
```

Такая логика необходима, но сама по себе создаёт дополнительные риски, которые нужно проектировать заранее.

Повторные попытки (Retries)

Проблема. Повторы без ограничений и без случайного разброса задержки (*jitter*) во время инцидента создают «шторм повторов». **Следствие.** Зависимость получает ещё больше трафика в момент деградации, а время восстановления увеличивается. **Что делать.** Повторять только временные ошибки, ограничивать число попыток, применять экспоненциальную задержку (*backoff*) и *jitter*, а также общий бюджет времени на операцию.

Идемпотентность (Idempotency)

Проблема. Повтор неидемпотентного запроса может выполнить бизнес-действие повторно. **Следствие.** Появляются дубликаты операций (например, двойное списание или двойная отправка заказа). **Что делать.** Использовать ключ идемпотентности (*idempotency key*) для операций с побочным эффектом и возвращать результат первой успешной обработки при повторе.

Автоматическое размыкание цепи (Circuit Breaker)

Проблема. Даже при наличии таймаутов трафик продолжает идти в недоступную зависимость. **Следствие.** Возникает каскадная деградация: upstream теряет ресурсы на заведомо безуспешные вызовы. **Что делать.** При превышении порога ошибок

размыкать цепь, временно отклонять вызовы или переключаться на резервный сценарий (**fallback**), затем выполнять ограниченные пробные запросы в промежуточном состоянии восстановления.

Передача дедлайна (Deadline Propagation)

Проблема. В цепочке сервисов внутренние таймауты часто не согласованы с исходным SLA запроса. **Следствие.** Внутренний вызов может ждать дольше, чем разрешено клиентом, и результат становится бесполезным. **Что делать.** Передавать единый дедлайн по всей цепочке, рассчитывать таймауты от оставшегося бюджета и логировать остаток времени в трассировке.

Практически это означает, что `timeout`, `retry`, `idempotency`, `circuit breaker` и `deadline propagation` следует рассматривать как единый контур устойчивости, а не как независимые опции.

Альтернатива — **асинхронное взаимодействие**: сервис не блокируется в ожидании ответа, а публикует событие или задачу в брокер/очередь. Это снижает связность по времени и лучше переживает пики нагрузки, но добавляет модель согласования состояния со временем (eventual consistency), необходимость дедупликации и контроль отставания потребителей очереди.

Во многих практических сценариях (обработка видео, генерация отчётов, интеграции с внешними системами, массовые рассылки) мгновенный ответ не нужен, поэтому асинхронная модель оказывается устойчивее и дешевле в эксплуатации.

Зачем асинхронность

- Развязка сервисов: отправитель не ждёт, пока получатель завершит обработку
- Устойчивость к пикам нагрузки: очередь буферизует всплески
- Повышение надёжности: повторная доставка, управляемые повторы и очередь проблемных сообщений (DLQ)
- Улучшение UX: пользователь получает быстрый **202 Accepted**, тяжёлая работа уходит в фон

Брокеры сообщений и очереди

- **Kafka** — журнал событий (event log), высокая пропускная способность, партиции, группы потребителей
- **RabbitMQ** — классические очереди задач, гибкая маршрутизация, подтверждения доставки
- **Pub/Sub** — одно событие читают несколько подписчиков (аналитика, уведомления, антифрод)
- **Task Queue** — одно задание исполняет один воркер (рендер PDF, отправка email)

Обмен файлами как асинхронный контракт

- Подходит для batch-интеграций между системами и организациями
- Примеры: S3/shared folder/SFTP, форматы CSV/Parquet/JSONL

- Важные практики: версионирование схемы, checksum, идемпотентная загрузка, дедупликация
- Риски: задержки, частичная загрузка, конфликт версий; нужны валидаторы и правила ретраев

Надёжность и согласованность

- Гарантии доставки: не более одного раза (at-most-once), не менее одного раза (at-least-once), практически однократно (effectively-once, через идемпотентность)
- Согласование состояния со временем (eventual consistency): состояние сходится не мгновенно, а через поток событий
- Подход Saga: длинные бизнес-процессы с компенсирующими действиями

Постскриптум: критерий архитектурного выбора

Итог первого модуля: большинство инструментов «работают» и действительно способны решить задачу. REST, gRPC, GraphQL, WebSocket, очереди и файловые интеграции — каждый подход может дать рабочий результат в правильном контексте.

Поэтому хороший архитектор не выбирает «любимый» инструмент заранее. Он выбирает инструмент под задачу, учитывая ограничения:

- требования к задержке и пропускной способности;
- характер пользовательского сценария (синхронный ответ или фоновая обработка);
- стоимость сопровождения и наблюдаемости;
- требования к надёжности, безопасности и эволюции контракта.

Рассмотренные в модуле принципы нужны именно для этого: принимать инженерные решения не по привычке, а по критериям конкретной системы.